ALLAB Yacine

Projet Caméra intelligente AXIS

Mise en place une architecture de caméra IP intelligente

TABLE DES MATIERES

Table des matières

INTRODUCTION 2
PROTOCOLE RTSP 2
ARCHITECTURE 3
MISE EN PLACE TECHNIQUE 4
INSTALLATION DE MEDIAMTX 4
MISE EN PLACE DE L'INFRASTRUCTURE 6
CONFIGURATION ET TESTS DE LA CAMERA 9
DEVELOPPEMENT DU SCRIPT PYTHON 10
CONCLUSION 13

INTRODUCTION

Ce projet a été réalisé dans le cadre de mon alternance au sein d'ArcelorMittal. Il répond à un besoin industriel de surveillance et d'analyse vidéo en temps réel. L'objectif est de mettre en place une architecture capable de récupérer, traiter et redistribuer un flux vidéo provenant d'une caméra réseau.

Dans le projet Caméra intelligente, nous avons mis en place une **architecture de diffusion vidéo** permettant de :

- Récupérer un flux vidéo provenant d'une caméra réseau Axis (IP privée : 10.189.20.110)
- Centraliser ce flux via un serveur de streaming MediaMTX déployé sous Docker
- Traiter le flux vidéo à l'aide d'un script Python permettant pour la suite d'intégrer des modèles de détections automatiques (IA)
- Ajouter dynamiquement un overlay (date, heure, identification caméra)
- Rendre le flux modifié accessible à un nouveau channel RTSP.

Ce projet permet d'aborder plusieurs notions techniques essentielles : la **gestion des flux vidéo** en **protocole RTSP**, la **configuration réseau** et **l'ouverture de ports**, la **conteneurisation avec Docker**, ainsi que le traitement d'image en temps réel avec **Python** et **OpenCV**. Il s'inscrit dans une logique d'architecture modulaire et professionnelle adaptée aux environnements de vidéosurveillance IP.

L'objectif final du projet, qui sera réalisé par un autre membre de l'équipe, consiste à **intégrer une intelligence artificielle capable d'analyser le flux vidéo** afin de sécuriser les opérations de levages. La solution mise en place dans ce projet constitue donc la base technique nécessaire à l'intégration de ce module d'analyse par IA.

PROTOCOLE RTSP

C'est quoi le RTSP ?

RTSP (Real Time Streaming Protocol) est un protocole réseau conçu pour le **contrôle et la transmission de flux vidéo en temps réel**, principalement pour les caméras IP et les systèmes de vidéosurveillance.

Dans notre architecture, la caméra envoie son flux **RTSP** vers le **serveur MediaMTX**. MediaMTX est configuré pour écouter sur le port 8554, qui est le port standard RTSP dans notre configuration Docker. Le flux est donc accessible via une adresse de type :

rtsp://IP_SERVEUR:8554/axis

Le script Python lit ce **flux RTSP**, applique un traitement (ajout d'un overlay avec date, heure et identification de la caméra), puis republie un **nouveau flux RTSP** sur le même serveur **MediaMTX**, également via le port **8554**, sous un chemin différent :

rtsp://IP_SERVEUR:8554/axis_overlay

Pourquoi ne pas utiliser RTMP ?

Le protocole **RTMP (Real-Time Messaging Protocol)** est principalement utilisé pour le **streaming sur Internet**, notamment vers des **plateformes de diffusion en direct.** Il fonctionne souvent en mode “push”, c’est-à-dire que la source envoie le flux vers un serveur distant.

Dans notre projet, le contexte est différent. Nous sommes dans un réseau local de vidéosurveillance. La caméra Axis ne diffuse pas nativement en RTMP, mais en RTSP. Utiliser RTMP aurait donc nécessité une conversion supplémentaire du flux, ainsi qu’une adaptation côté serveur

Le choix du RTSP :

Le choix du RTSP repose sur deux éléments principaux. Premièrement, la compatibilité avec la caméra Axis, qui émet en RTSP. Deuxièmement, la compatibilité avec MediaMTX, configuré pour exposer le service RTSP sur le port 8554 via Docker.

Dans cette architecture, le flux vidéo suit un chemin précis. La caméra Axis envoie le flux RTSP vers le serveur MediaMTX. Ce flux est ensuite lu par le script Python qui applique un traitement. Enfin, le flux modifié est renvoyé vers MediaMTX dans un second channel.

Le port utilisé pour la diffusion RTSP est le port 8554, exposé via Docker. Ce port permet aux clients comme VLC ou un serveur VMS d’accéder aux flux vidéo.

ARCHITECTURE

L’infrastructure du projet repose sur une **architecture réseau locale** intégrant une **caméra IP**, un **serveur de reception/diffusion vidéo** et un **poste de traitement**. L’objectif est d’assurer la réception, le traitement et la redistribution d’un flux vidéo en temps réel de manière stable et structurée.

Caméra Axis (RTSP)
↓
MediaMTX (channel 1 : axis)
↓
Script Python (OpenCV + FFmpeg ❼ traitement + overlay)
↓
MediaMTX (channel 2 : axis_overlay)
↓
Client VLC

Composants utilisés :

1. **Caméra réseau :**
   - Modèle de caméra : AXIS P1465-LE
   - IP : 10.189.20.110 (configuré de base)
   - Protocole utilisé : RTSP
   - URL utilisée : rtsp://root:PASSWORD@10.189.20.110/axis-media/media.amp

2. **Serveur vidéo (MediaMTX)**

Notre serveur vidéo utilisera MediaMTX déployé sur Docker dans notre serveur physique.

- Serveur de streaming RTSP
- Déployé via Docker
- Rôle : proxy et redistribution des flux vidéo

L'utilisation de **Docker** permet d'isoler le service, de simplifier son déploiement et de faciliter la gestion des ports. Le port 8554 du conteneur est exposé vers l'extérieur afin de permettre l'accès au service RTSP. Cela signifie que toute machine du réseau peut accéder au flux via l'adresse IP du serveur suivie du port 8554.

3. **Script Python**

- OpenCV (traitement vidéo)
- FFmpeg (ré-encodage et publication RTSP)
- Python

. Ce script aura pour rôle de lire le flux vidéo modifié depuis MediaMTX, d'y ajouter un overlay dynamique (date en temps réel, nom de la caméra, emplacement de l'usine). Cela nous permettra par la suite d'y ajouter un modèle d'IA en temps réel sur la caméra.

4. **Réseau**

D'un point de vue réseau, tous les équipements doivent être situés dans le même sous-réseau ou disposer d'un routage autorisant la communication.

MISE EN PLACE TECHNIQUE

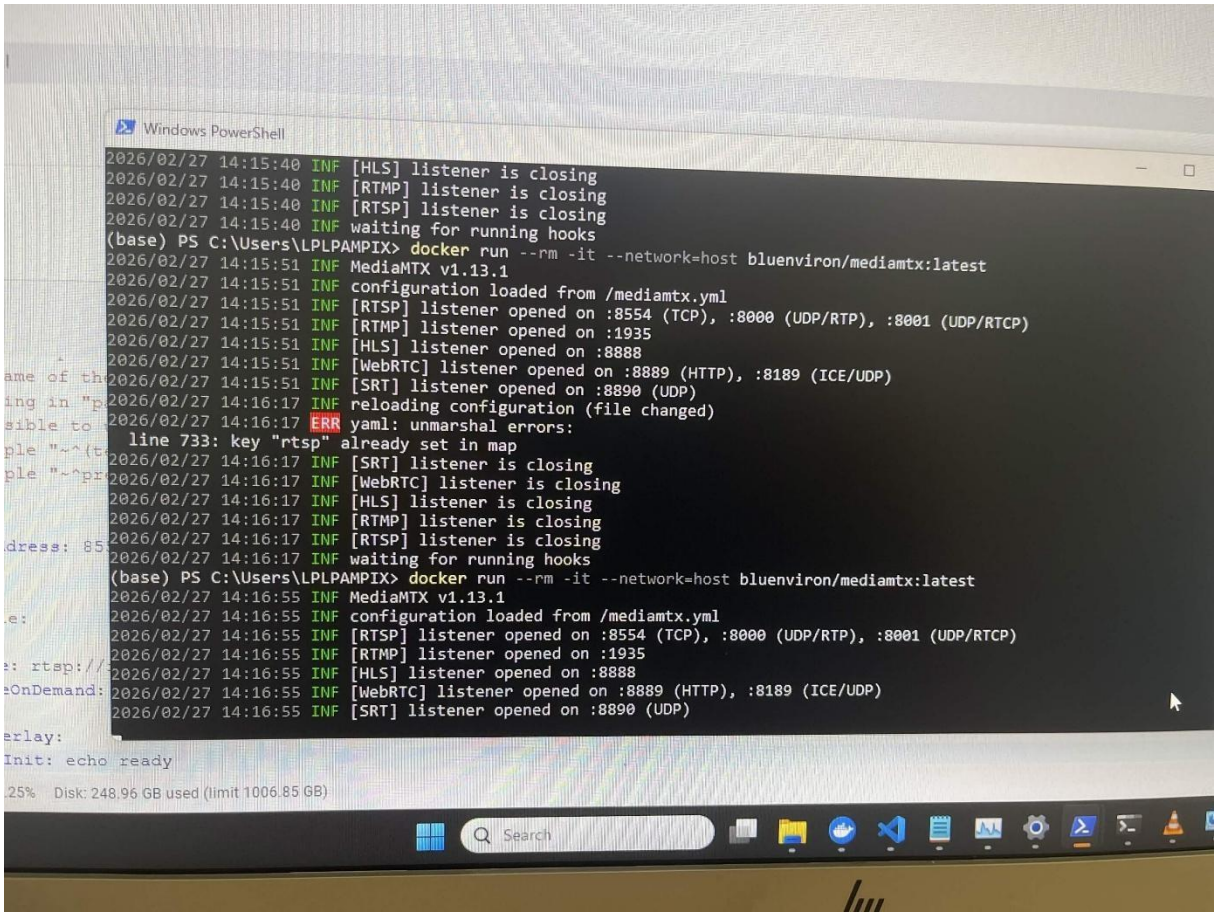
INSTALLATION DE MEDIAMTX

Voici le lien de la documentation de MediaMTX pour l'installer et le conteneuriser : https://mediamtx.org/

Suivez les étapes de la documentation (environ 10-15 minutes).

MediaMTX a été déployé via Docker sur le poste de travail. Le conteneur a été lancé avec la commande suivante : docker run –rm -it –network=host bluenviron/mediamtx:latest

Je lance l'image de MediaMtx via docker afin de créer un conteneur. :

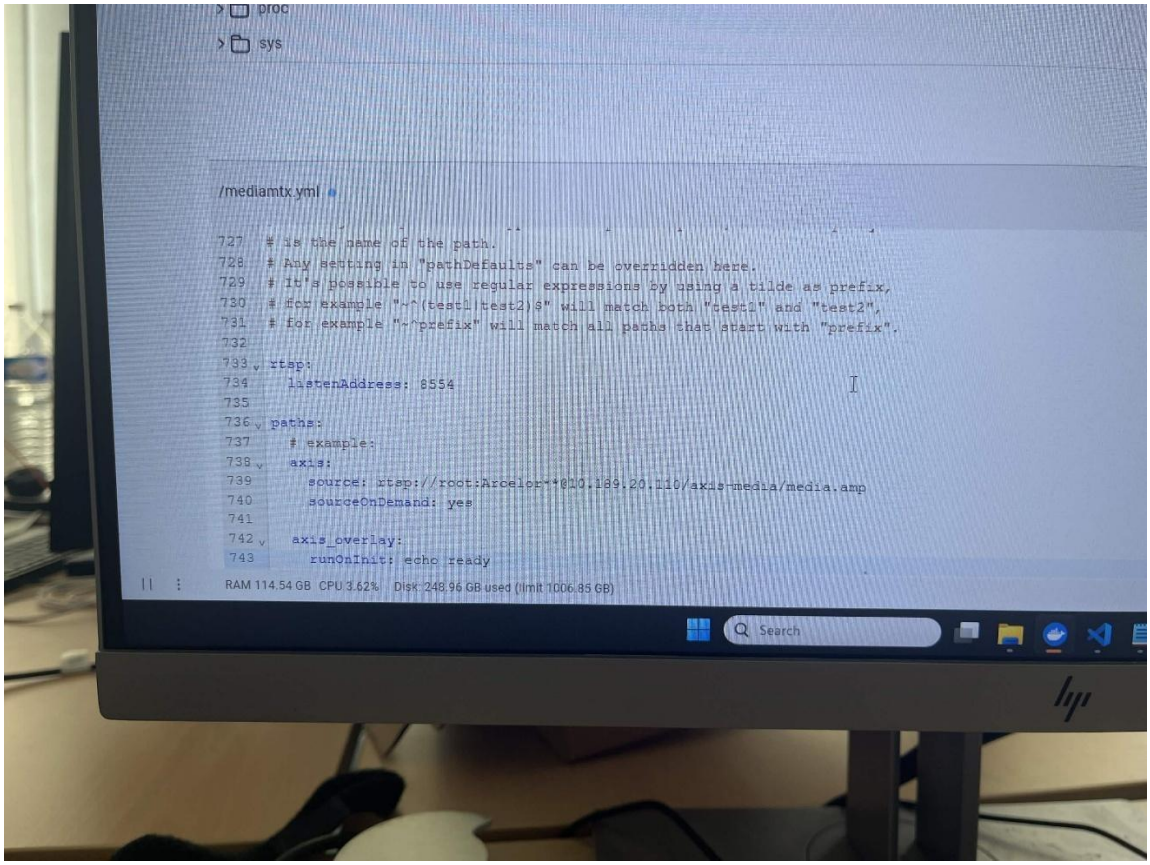


Dans notre conteneur Docker, on se rend dans **le fichier de configuration mediamtx.yml.** On ajoutera d'une part la source de notre caméra (axis). Et d'une autre part ajouter la source de l'overlay qui exportera notre flux vidéo (axis_overlay).

En l'occurrence

paths: axis:
    source: rtsp://root:PASSWORD@10.189.20.110/axis-media/media.amp
sourceOnDemand: yes

axis_overlay: {}



❾ Cette étape est primordiale car sans ça le serveur vidéo ne détecte pas le flux de la caméra et ne peux pas l'exporter.

MISE EN PLACE DE L'INFRASTRUCTURE

L'architecture physique mise en place possède un ordinateur, une caméra Axis, un boitier injecteur PoE permettant d'alimenter la caméra via un câble physiques Ethernet.

Qu'est-ce que le POE ?

Le **Power over Ethernet (PoE)** est une technologie qui permet de transmettre à la fois des **données réseau et de l'électricité** à travers un seul câble Ethernet (RJ45). Cela signifie qu'un appareil peut fonctionner sans alimentation électrique séparée, uniquement grâce au câble réseau.

Le PoE est très utilisé pour des équipements comme les caméras de surveillance IP, les téléphones VoIP, les bornes Wi-Fi ou encore certains systèmes domotiques. Il simplifie l'installation, réduit le nombre de câbles nécessaires et permet de placer les appareils dans des endroits où il n'y a pas de prise électrique à proximité.

Architecture physique :

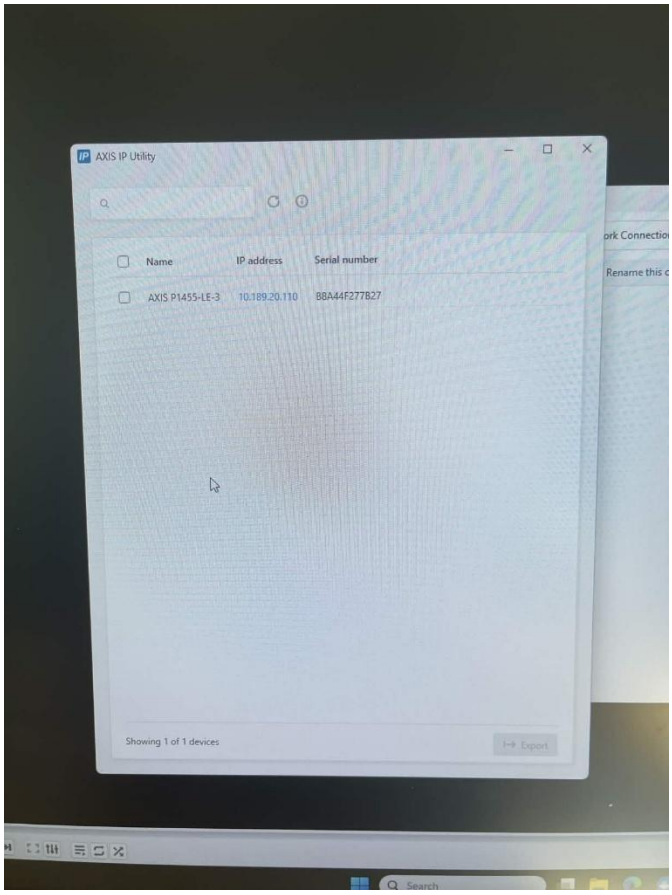
La caméra Axis ne possède pas d'alimentation secteur classique, elle fonctionne en PoE, ce boitier possède 2 ports :

- *Data In :* ce port est relié au PC et renvoie les **données réseau** de la caméra au PC (qui fait office de serveur vidéo).

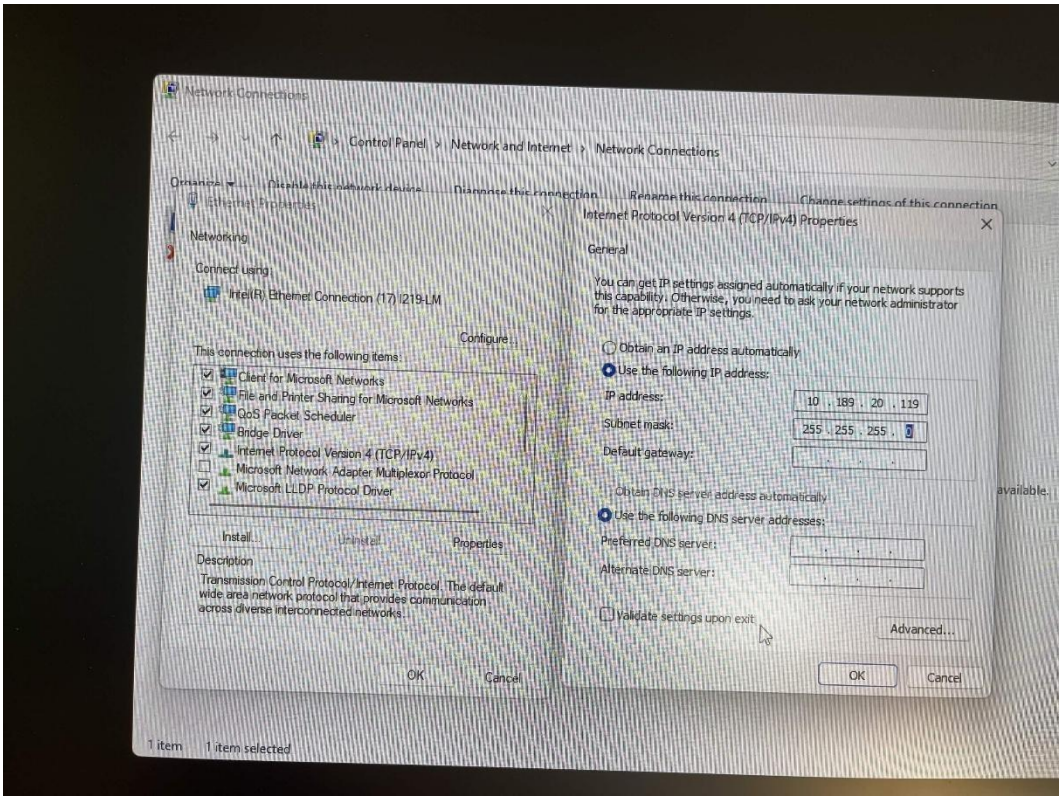
- *Data & Power Out :* ce port fournit à la fois **l'alimentation électrique** de la caméra et les **données réseau** vers la caméra.

Configuration réseau :

La caméra a été configurée avec une adresse IP privée fixe : 10.189.20.110



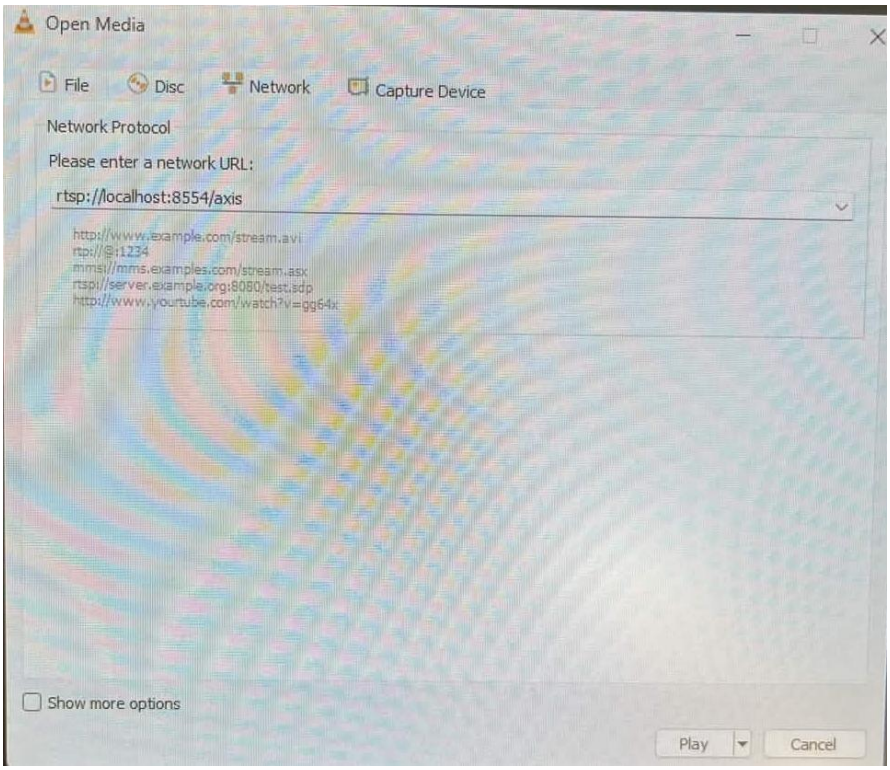
Le serveur physique se trouvait sur le même sous-réseau afin de pouvoir communiquer directement avec la caméra via RTSP. Je l'ai alors mis en 10.189.20.119



Le protocole utilisé pour récupérer le flux vidéo est **RTSP (Real Time Streaming Protocol)**, par défaut accessible via cette URL sur les caméras AXIS :

rtsp://IP_CAMERA/axis

Ce flux a été testé initialement via VLC afin de valider la connectivité réseau et le bon fonctionnement matériel.



CONFIGURATION ET TESTS DE LA CAMERA

Une fois avoir entré l'adresse du flux de la caméra sur VLC, vous aurez accès à l'image de la caméra en temps réel. Le flux de la caméra a été testé à l'aide du logiciel VLC avec l'URL RTSP suivante :

rtsp://root:PASSWORD@10.189.20.110/axis-media/media.amp

Cette étape a permis de valider la connectivité réseau et le bon fonctionnement de la caméra avant intégration dans MediaMTX.

Nous arrivons donc à recevoir le flux vidéo de la caméra. L'objectif maintenant est de faire passer ce flux vers le serveur vidéo MediaMtx, développer un script Python qui ajoute un overlay sur le flux pour qu'enfin le serveur vidéo renvoie le flux vidéo modifié.

DEVELOPPEMENT DU SCRIPT PYTHON

Le script commence par lire le flux RTSP brut diffusé par MediaMTX depuis Axis.Ensuite, Il récupère ensuite les images une par une à l'aide d'OpenCV. Sur chaque image, deux superpositions (overlays) sont ajoutées : un texte fixe permettant d'identifier la caméra, ainsi qu'un texte dynamique affichant la date et l'heure. Une fois l'image modifiée, elle est envoyée à FFmpeg sous forme de flux vidéo brut.

Dans la variable LABEL, j'ai défini le texte qui sera affiché après l'ajout de l'overlay : CAMERA 1 : VUE PARC A MITRAILLE.

Deux variables principales sont utilisées :

- IN_URL : flux d'entrée (axis)
- OUT_URL : flux de sortie (axis_overlay)

FFmpeg joue un rôle essentiel dans l'architecture. Il permet de convertir les images traitées par Python en flux vidéo compressé au format H.264, puis de les envoyer vers MediaMTX en RTSP.

Sans FFmpeg, il serait impossible de republier le flux traité.L'option -rtsp_tranport tcp a été utilisée afin d'éviter les blocages liés au pare-feu.

```python
import cv2
import subprocess
import time
import shutil

IN_URL = "rtsp://127.0.0.1:8554/axis"
OUT_URL = "rtsp://127.0.0.1:8554/axis_overlay"
LABEL = "CAMERA 1 : VUE PARC A MITRAILLE"

def main():
    ffmpeg = shutil.which("ffmpeg")
    if not ffmpeg:
        raise RuntimeError("ffmpeg introuvable. Installe-le dans l'environnement conda.")

    cap = cv2.VideoCapture(IN_URL, cv2.CAP_FFMPEG)
    if not cap.isOpened():
        raise RuntimeError(f"Impossible d'ouvrir {IN_URL}")

    ok, frame = cap.read()
    if not ok or frame is None:
        raise RuntimeError("Impossible de lire la première frame")

    h, w = frame.shape[:2]
    fps = cap.get(cv2.CAP_PROP_FPS)
    if not fps or fps != fps or fps < 1:
        fps = 25
    fps = int(round(fps))

    cmd = [
        ffmpeg,
        "-loglevel", "error",
        "-f", "rawvideo",
        "-pix_fmt", "bgr24",
        "-s", f"{w}x{h}",
        "-r", str(fps),
        "-i", "-",
        "-an",
        "-c:v", "libx264",
        "-preset", "veryfast",
        "-tune", "zerolatency",
        "-rtsp_transport", "tcp",
        "-f", "rtsp",
        OUT_URL,
```

```python
    proc = subprocess.Popen(cmd, stdin=subprocess.PIPE, stderr=subprocess.PIPE)

    try:
        while True:
            dt = time.strftime("%Y-%m-%d %H:%M:%S")

            cv2.putText(
                frame,
                LABEL,
                (20, h - 60),
                cv2.FONT_HERSHEY_SIMPLEX,
                0.9,
                (0, 255, 0),
                2,
                cv2.LINE_AA
            )

            cv2.putText(
                frame,
                dt,
                (20, h - 20),
                cv2.FONT_HERSHEY_SIMPLEX,
                0.8,
                (0, 255, 0),
                2,
                cv2.LINE_AA
            )

            try:
                proc.stdin.write(frame.tobytes())
            except BrokenPipeError:
                err = proc.stderr.read().decode("utf-8", errors="ignore")
                raise RuntimeError("FFmpeg s'est arrêté. Erreur :\n" + err)

            ok, frame = cap.read()
            if not ok or frame is None:
                raise RuntimeError("Perte du flux axis pendant la lecture")

    finally:
        cap.release()
        try:
            if proc.stdin:
                proc.stdin.close()
        except Exception:
            pass
        proc.wait()
```

Le flux final peut ensuite être relu dans VLC ou injecté dans un VMS avec l'adresse rtsp://127.0.0.1:8554/axis_overlay.

Pour que ce script fonctionne, plusieurs points ont été mis en place. D'abord, MediaMTX a été configuré avec deux paths : axis pour récupérer le flux brut depuis la caméra Axis, et axis_overlay pour recevoir le flux modifié. Ensuite, FFmpeg a été installé dans l'environnement Conda afin qu'il soit accessible depuis Python. Enfin, la publication RTSP a été forcée en TCP avec l'option -rtsp_transport tcp, ce qui a permis d'éviter les blocages liés au pare-feu Windows et aux restrictions réseau de la machine.

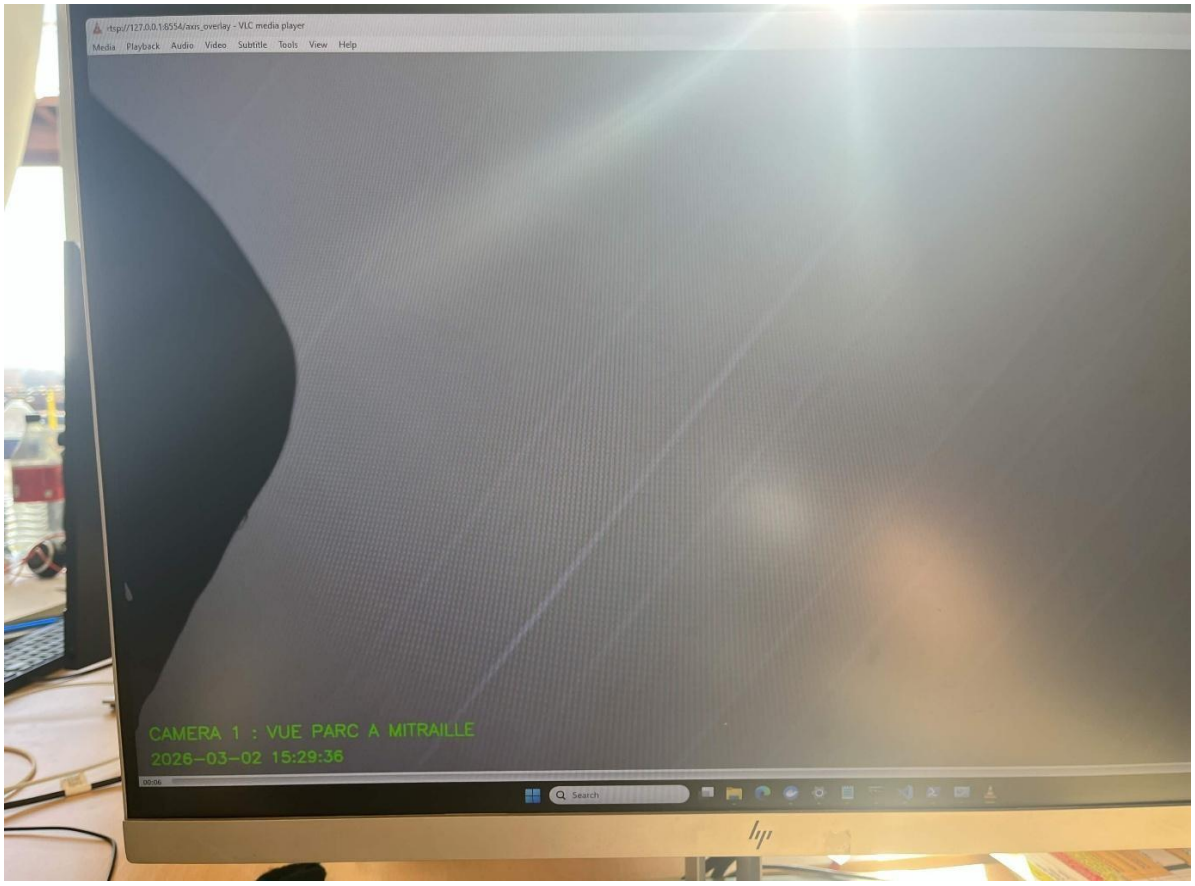
RESULTATS OBTENUS

Nous avons lancé le script Python qui ajoutait l'overlay sur le script vidéo en passant par le serveur MediaMtx.

Le système mis en place permet d'obtenir deux flux distincts :

- Un flux brut (axis) correspondant à la caméra (AVANT):

Un flux modifié (axis_overlay) avec overlay (APRES):



Le résultat affiche correctement :

- Le label : CAMERA 1 : VUE PARC A MITRAILLE
- La date et l'heure en temps réel

Le flux est stable et exploitable pour une future analyse automatisée.

CONCLUSION

Ce projet a permis de mettre en place une architecture complète de gestion de flux vidéo IP. Il a permis de maîtriser l'utilisation du protocole RTSP, le déploiement d'un serveur de streaming avec Docker, ainsi que le traitement vidéo en temps réel avec Python. L'objectif de ce projet a été de mettre en place une architecture complète et réutilisable de gestion de flux vidéo IP. L'architecture sera réutilisable dans le cadre de tous les projets de vision industrielle de LPL pour faire tourner de la détection de lux automatique sur des caméras vidéo.